CST1450 CW2 - Dijkstra's Algorithm Report

1. Running the code

Required libraries:

json
pathlib
dataclasses

The project consists of four Python 3 files and a folder containing JSON files. The first file, named "Edge.py", has a class to define an edge for the graph; the second file, named "JSON_to_graph.py", has functions for conversion to a graph class, which is defined in the file "Graph.py". The "Main.py" file, which is the last Python 3 file, uses all the functions from the previous files to perform Dijkstra's algorithm.

To run the algorithm, open a terminal, navigate to the project directory, and execute:

python3 Main.py

The program will run for the 6 JSON files, 5 provided and 1 defined. The program will take "A" as the start vertex and "Z" as the end vertex. If the user wants to change the start or end vertex, they will need to open "Main.py" and update the vertices (in quotation marks) shown in the screenshot below.

```
# Storing the path and distance using dijkstra's algorithm.
result_path, distance = graph.dijkstra_algorithm( start_vertex: "A",  end_vertex: "Z")
```

If the user wishes to run the program on another graph, they will need to add the file name to the list of paths in "Main.py", maintaining the structure in the screenshot.

```
paths = [
    BASE_DIR / "JSON files/graph_1.jsn",
    BASE_DIR / "JSON files/graph_2.jsn",
    BASE_DIR / "JSON files/graph_3.jsn",
    BASE_DIR / "JSON files/graph_4.jsn",
    BASE_DIR / "JSON files/graph_5.jsn",
    BASE_DIR / "JSON files/created_graph.jsn"
]
```

2. *JSON files: graphs results*

The first graph runs without any errors. The graph is correctly defined, and all vertices are connected. The output correctly lists all the vertices from the graph and gives the correct shortest path from “A” to “Z”.

```
Graph 1:

Vertices: ['Z', 'C', 'A', 'B', 'D']

Shortest path: ['A', 'B', 'C', 'Z']

Total distance: 4
```

The second graph does not return any path, because the graph is wrongly defined. The edge from “A” to “B” is defined more than 1 times, so the code raises an exception for it.

```
Graph 2:

❌Error❌
The edge B - A already exists.
```

The third graph lists all the vertices, so the graph is correctly defined, but it does not contain the vertex “Z”, so when we are trying to find the path from “A” to “Z”, the program raises an exception for it, saying the vertex does not exist in the graph.

```
Graph 3:

Vertices: ['C', 'A', 'B', 'D', 'E']

❌Error❌
The end vertex Z does not exist in the graph.
```

The fourth graph returns the complete shortest path from “A” to “Z”. Same as the first graph, it correctly defines the graph as having both “A” and “Z” vertices, and the graph is valid.

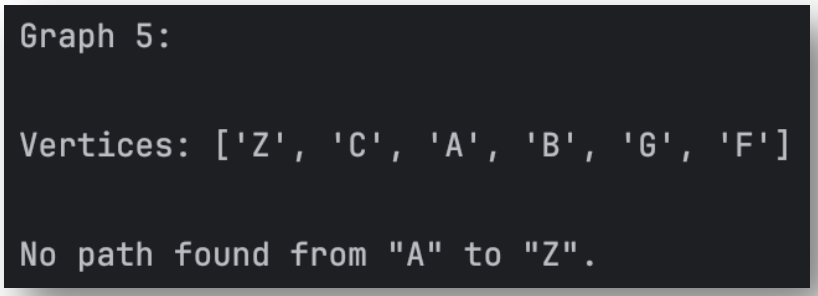
```
Graph 4:

Vertices: ['B', 'H', 'S', 'T', 'V', 'A', 'F', 'E', 'U', 'C', 'J', 'D', 'L', 'W', 'I', 'N', 'Q', 'P', 'O', 'Z', 'K', 'G', 'R', 'X', 'Y', 'M']

Shortest path: ['A', 'B', 'E', 'I', 'F', 'G', 'L', 'R', 'P', 'T', 'V', 'X', 'Y', 'Z']

Total distance: 18
```

The fifth graph is also a valid example of a graph, but we are searching for the path from “A” to “Z”, so in this case, those vertices are not connected to each other. Because of that, Dijkstra’s algorithm returns an infinite value, which is then displayed as a statement that states that no path was found.



In the second graph, we can observe that the program was stopped right in the beginning, because of an early bug in the definition of one of the edges of the graph. We must catch those wrongs as soon as possible, because if we let them slide, they can cause many other difficulties in the future.

Even though the third graph’s edges and the graph are proven to be valid, we must track whether the vertices that we input into the graph are part of the graph.

3. Improvements

When it comes to improvements, we must be very cautious. Some of the changes can improve our work very much, but we always need to pay attention to the fact that we can also worsen our work.

The first, most important change I would make is to store the edges and vertices in the graph through the adjacency list represented by a dictionary. This small change has a significant impact on the algorithm. One of the main parts of the algorithm is to go through all the neighbours of one vertex. This operation requires the "get_neighbours" function, and look at the difference in the screenshots below, how big a change this makes in the function.

Default

```python
# Using list a storage
def get_neighbours(self, vertex: str):

    # Set to store the neighbours (no duplicates)
    neighbours = set()

    # For loop to find the neighbours in the list of edges
    for edge in self.list_of_edges:
        # Checking the start and end vertex for matches
        if vertex == edge.start_vertex:
            neighbours.add(edge.end_vertex)
        elif vertex == edge.end_vertex:
            neighbours.add(edge.start_vertex)

    # Returning the neighbours
    if neighbours:
        return list(neighbours)
    else:
        return False
```

Improved

```python
# Using Adjacency list as storage (dictionary)
def get_neighbours(self, vertex: str):

    return [neighbour for neighbour, d in self.adj_list.get(vertex, [])]
```

The difference between those two functions does not need to be noticeable for smaller graphs, but as the number of vertices in the graph increases, the complexity of these functions varies so much.

With the change in the structure of storing the edges, it also improves the complexity of other functions, such as “are_adjacent” or “list_vertices”.

The second improvement I would make is to use the heaps. In Dijkstra’s algorithm function, we can use them to reduce the complexity of searching the edges to check which path is the most efficient. Heaps take care of sorting our adjacency list, so the time complexity of finding the edge which cost the least is improved.